

Mobile AI Project



FocusFlow+

AI-guided focus sessions based on task, energy, and available time

Student Jaouher Chouchane

Profile IT/BA Junior Student

Instructor Prof. Najet Boughanmi

Technology Stack

Mobile development	Android, Kotlin, Jetpack Compose
Architecture	MVVM-inspired structure, repositories, domain models
Artificial intelligence	Gemini API through Retrofit and OkHttp
Persistence	Room Database, SessionDao, local session history
Project workflow	Git, GitHub, Gradle, Android Studio

Abstract

FocusFlow+ is a native Android application that helps users turn a task into a realistic focus session. The app collects a short task description, the user's current energy level, and the time available for work. These inputs are sent to the Gemini API, which returns a structured recommendation containing focus duration, break duration, a session structure, health tips, and a motivational message.

The project was implemented with Kotlin, Jetpack Compose, Retrofit, OkHttp, and Room Database. It follows an MVVM-inspired architecture where screens, ViewModels, repositories, remote API access, and local persistence are separated. The final MVP covers the complete workflow: input collection, AI plan generation, timer execution, summary display, and saved history.

Keywords: Android, Kotlin, Jetpack Compose, Gemini API, Room Database, MVVM, focus timer, productivity.

Contents

Abstract	i
1 Introduction	1
1.1 Project Context	1
1.2 Project Objective	1
1.3 MVP Definition	1
2 Problem Analysis	1
2.1 Target Users	2
3 Proposed Solution	2
3.1 How the Solution Fixes the Problem	2
3.2 Scope Boundaries	3
4 Requirements	3
4.1 Functional Requirements	3
4.2 Non-Functional Requirements	3
5 Analysis and Design	4
5.1 Use Case Diagram	4
5.2 System Architecture	5
5.3 Database Design	6
5.4 Class Diagram	7
6 Implementation	7
6.1 User Input and Planning Screen	7

6.2	Gemini Integration	8
6.3	Timer Execution	8
6.4	Summary and History	9
6.5	Implementation Flow	10
7	Testing and Validation	10
7.1	Observed Result	11
8	Limitations and Future Improvements	11
8.1	Current Limitations	11
8.2	Future Work	11
9	Conclusion	12
	References	12

1 Introduction

1.1 Project Context

Many users begin a study or work session with a task in mind, but without a clear way to divide their time. The first barrier is often not the task itself, but the planning step before starting: deciding how long to focus, when to take a break, and how to adapt the session to the user's current energy.

Standard timer applications are useful, but they normally apply the same structure for every situation. A fixed timer can ignore whether the task is demanding, whether the user is tired, and how much time is actually available. FocusFlow+ addresses this limitation by placing an AI planning step before the timer begins.

1.2 Project Objective

The objective of FocusFlow+ is to build a mobile assistant that supports a complete focus workflow:

- collect the user's task, energy level, and available time;
- generate a personalized focus plan using Gemini;
- guide the user through a focus/break timer;
- save the session locally using Room Database;
- allow the user to review summaries and history.

1.3 MVP Definition

In this report, MVP means Minimum Viable Product. For FocusFlow+, the MVP is the first complete version that proves the idea works without adding advanced features. The MVP includes planning, AI recommendation, timer execution, local storage, summary, and history. It does not yet include authentication, cloud synchronization, notifications, or user profiles.

Table 1: Technology overview

Technology	Role in the project
Kotlin	Main language for Android logic and ViewModel implementation.
Jetpack Compose	Declarative toolkit used to build the mobile interface.
Gemini API	External AI service used to generate a focus recommendation.
Retrofit/OkHttp	Networking tools used to call the Gemini REST endpoint.
Room Database	Local persistence layer used for planned and completed sessions.
GitHub	Version control and project sharing.

2 Problem Analysis

Real scenario: "I know what I need to do, but I do not know how to turn it into a realistic work session."

This problem is common in real life. A student may need to revise for an exam, complete an assignment, or prepare for a test. A worker may need to finish a report or organize a difficult task. In both cases, the user can know the goal but still feel blocked by the planning step.

The difficulty comes from three constraints happening at the same time: the task has a certain level of effort, the user has a limited amount of time, and the user’s energy is not always stable. A timer that ignores these constraints can become either too strict or too weak.

Table 2: Main problems identified

Problem	Description	Impact on the user
Planning overload	The user has to decide the session duration, break time, and starting point.	Time is wasted before work begins.
Fixed timer logic	A static 25/5 timer treats every task and every user state the same way.	The plan may be unrealistic for the task or energy level.
Energy mismatch	Low-energy users and high-energy users may need different structures.	A session can become discouraging or inefficient.
No local memory	Without saved sessions, the app cannot show history or progress.	The user cannot review previous work easily.

2.1 Target Users

The application targets users who need a simple way to move from intention to action. This includes students planning revision sessions, junior professionals managing focused tasks, and busy users who want a lightweight productivity assistant instead of a complex task management platform.

3 Proposed Solution

FocusFlow+ solves the problem by using AI as a decision-support layer. The user does not manually guess the plan. Instead, the app asks for the minimum useful context and transforms it into a guided focus session.



Figure 1: High-level workflow of FocusFlow+

3.1 How the Solution Fixes the Problem

The application improves the focus planning process in four practical ways:

1. **It reduces the planning burden.** The user only provides context; the app proposes a structure.
2. **It adapts to available time.** The generated focus and break durations must fit inside the selected time.
3. **It considers energy level.** A tired user receives a more gentle plan, while a high-energy user can receive a deeper focus block.

4. **It keeps progress visible.** Saved sessions are available in the history screen after closing and reopening the app.

3.2 Scope Boundaries

The current version focuses on the core productivity loop. Authentication, cloud synchronization, reminders, detailed analytics, and multi-device access are intentionally kept outside the MVP. This keeps the project realistic, testable, and focused on the AI-assisted timer concept.

4 Requirements

4.1 Functional Requirements

Functional requirements describe what the application must do.

Table 3: Functional requirements

ID	Requirement	Description
FR1	Enter task description	The user writes the task they want to work on.
FR2	Select energy level	The user selects their current energy from low to high.
FR3	Select available time	The user chooses how much time is available for the session.
FR4	Generate AI plan	The app calls Gemini and receives a structured focus recommendation.
FR5	Display recommendation	The app displays focus minutes, break minutes, session structure, tips, and motivation.
FR6	Run timer	The user can start, pause, reset, move to break, and complete the session.
FR7	Save session locally	The app saves planned and completed sessions using Room Database.
FR8	View summary and history	The user can review the completed session and previous sessions.

4.2 Non-Functional Requirements

Non-functional requirements describe how well the application should behave.

Table 4: Non-functional requirements

Quality attribute	Requirement
Usability	The interface should be clear, mobile-first, and easy to understand without training.
Performance	The app should avoid unnecessary API calls and keep the local database lightweight.
Reliability	The app should display clear errors for missing API key, empty task, or failed AI response.
Maintainability	The code should separate UI, ViewModels, repositories, API models, and database entities.
Security	The API key should not be pushed to GitHub; it should be stored locally during development.
Extensibility	The design should allow future features such as profiles, analytics, reminders, or backend API proxy.

5 Analysis and Design

5.1 Use Case Diagram

The use case diagram identifies the main actor and supporting systems. The user interacts with the application, while Gemini and Room Database are used internally by the app.

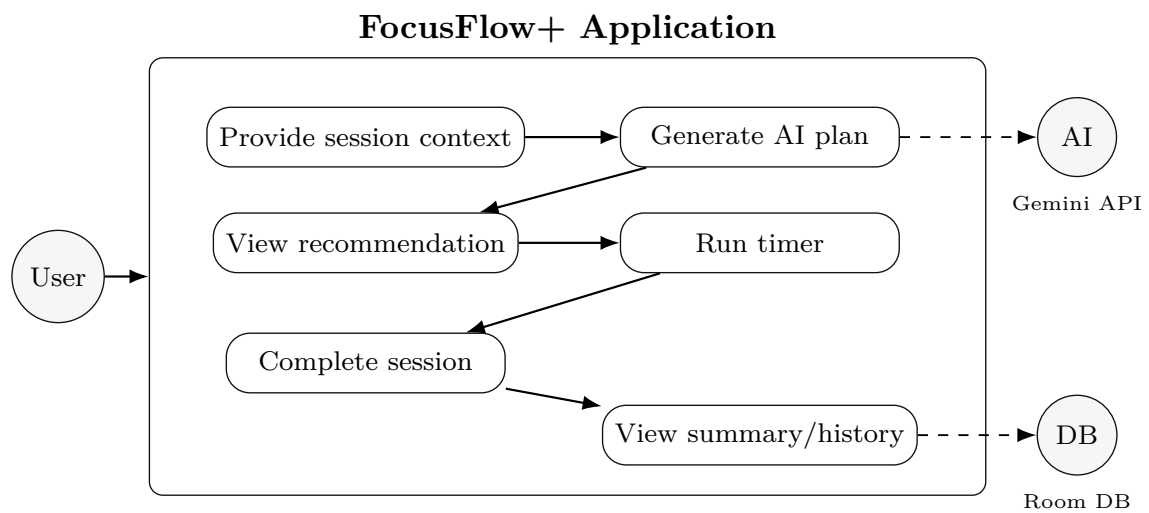


Figure 2: Use case diagram of the main interactions

The user is linked to *Complete session* because the user presses the completion action in the timer screen. The database is not directly controlled by the user. It saves or retrieves session data when the application needs it.

5.2 System Architecture

FocusFlow+ uses a layered architecture inspired by MVVM. Each layer has a clear responsibility, which keeps the project easier to understand and modify.

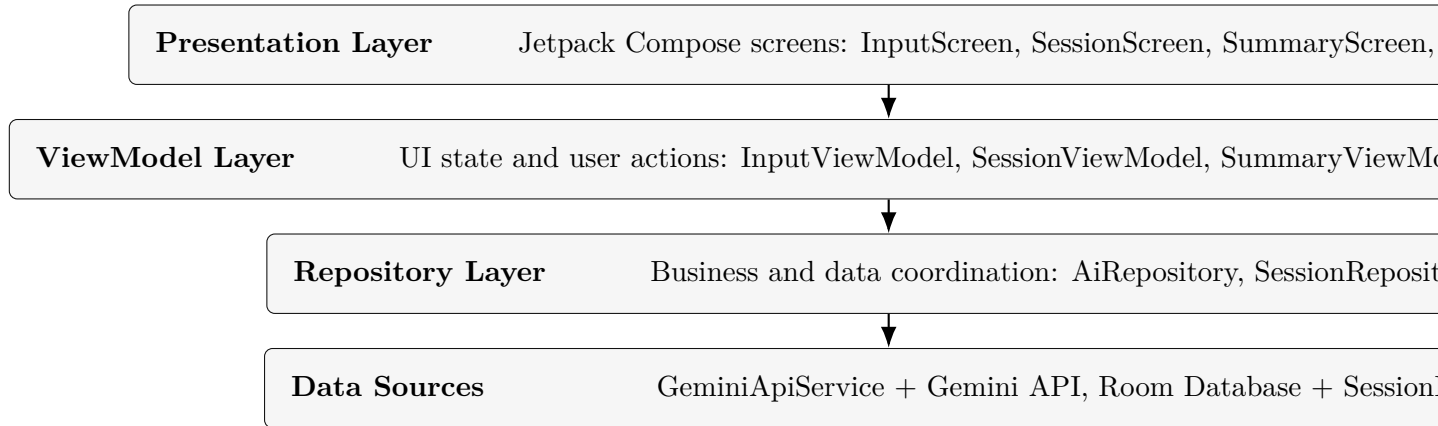


Figure 3: System architecture layers

The UI does not call Gemini or the database directly. Screens communicate with ViewModels, ViewModels use repositories, and repositories communicate with external or local data sources.

5.3 Database Design

Room Database is used to keep session data after the app is closed. The application stores the user's task, the generated recommendation, completion status, and timestamps.

Table 5: Session entity design

Field	Type	Purpose
id	Long	Primary key generated by Room.
taskDescription	String	User task, for example "Testing myself".
energyLevel	Int	Energy selected by the user.
availableMinutes	Int	Total time selected before generation.
focusMinutes	Int	Focus duration returned by Gemini.
breakMinutes	Int	Break duration returned by Gemini.
sessionStructure	String	Text explaining how the session should be divided.
healthTips	String	Stored tips, usually serialized as text.
motivationalMessage	String	Short motivation returned by Gemini.
createdAt	Long	Creation timestamp.
completedAt	Long?	Completion timestamp when the session is finished.
wasCompleted	Boolean	Indicates whether the session was completed.

Table 6: Main SessionDao operations

Operation	Role
insertSession(session)	Saves a new planned session after an AI recommendation is generated.
updateSession(session)	Updates a saved session, for example when it is completed.
getAllSessions()	Retrieves saved sessions for the history screen.
getSessionById(id)	Loads one session for the timer or summary screen.
getCompletedSessionCount()	Counts completed sessions for statistics.
getTotalFocusMinutes()	Calculates total completed focus time.

5.4 Class Diagram

The class diagram groups the main classes by responsibility. It shows how the ViewModels connect to repositories, how the AI layer is separated from the UI, and how local storage is accessed through Room.

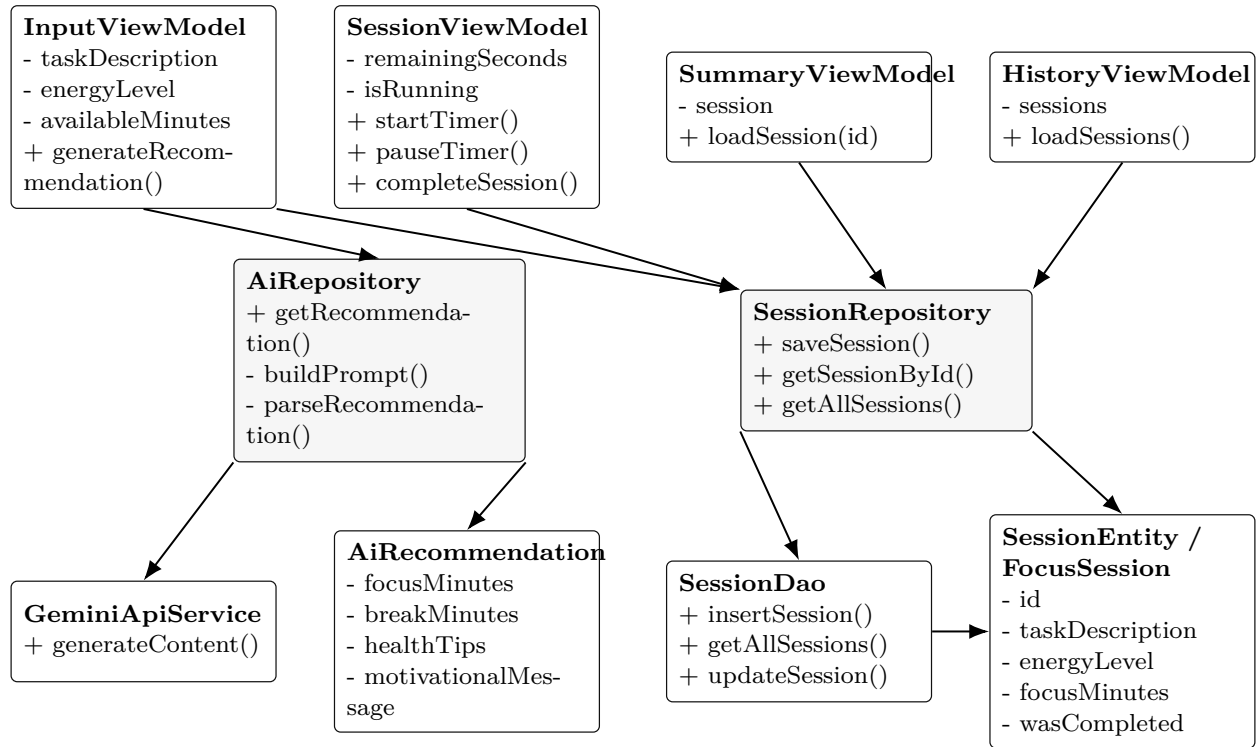


Figure 4: Class diagram of the main implemented classes

In the diagram, the minus sign indicates a private attribute, and the plus sign indicates a public method. For example, `generateRecommendation()` is public because the input screen can trigger it, while values such as `taskDescription` represent internal state managed by the ViewModel.

6 Implementation

6.1 User Input and Planning Screen

The input screen is the starting point of the workflow. It collects the user's task, energy level, and available time. These inputs are managed by **InputViewModel**, which validates the task and triggers the generation process.

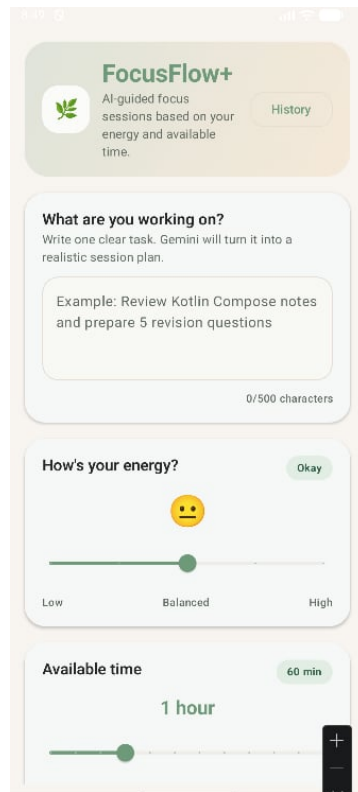


Figure 5: Input screen: task, energy level, and available time

6.2 Gemini Integration

The AI implementation is handled through `AiRepository`. The repository builds a prompt from the user input, calls Gemini through `GeminiApiService`, and parses the returned JSON-like structure into an `AiRecommendation` object.

Listing 1: Simplified structure of the AI recommendation model

```
data class AiRecommendation(
    val focusMinutes: Int,
    val breakMinutes: Int,
    val sessionStructure: String,
    val healthTips: List<String>,
    val motivationalMessage: String,
    val rawResponse: String
)
```

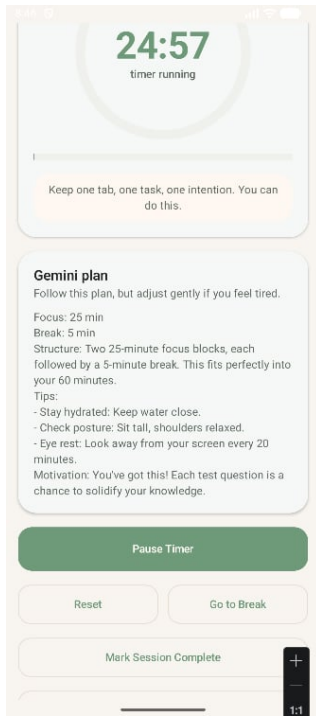
The API key is not stored in the source code. During development, it is read from `local.properties`, which is ignored by Git. This avoids pushing the key to GitHub.

Listing 2: Local Gemini configuration example

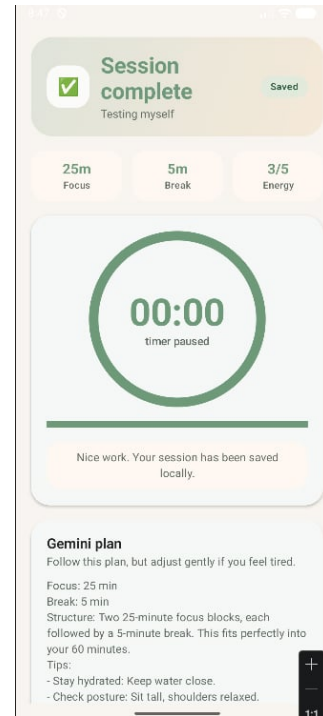
```
GEMINI_API_KEY=your_real_key_here
GEMINI_MODEL=gemini-2.5-flash
```

6.3 Timer Execution

After receiving the recommendation, the user enters the session screen. The timer is controlled by `SessionViewModel`, which manages whether the timer is running, paused, reset, or completed. The AI plan is displayed next to the timer so that the user understands the proposed session structure.



(a) Timer running

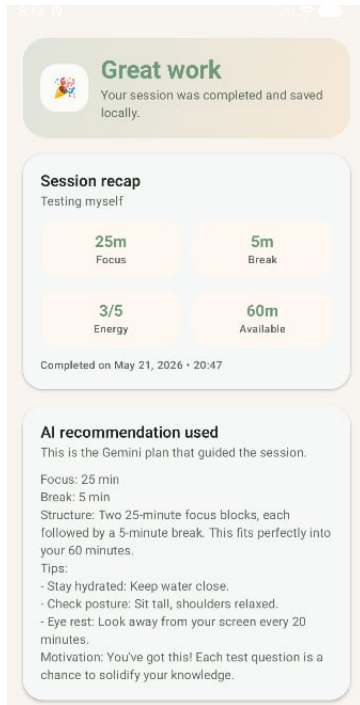


(b) Session marked as saved

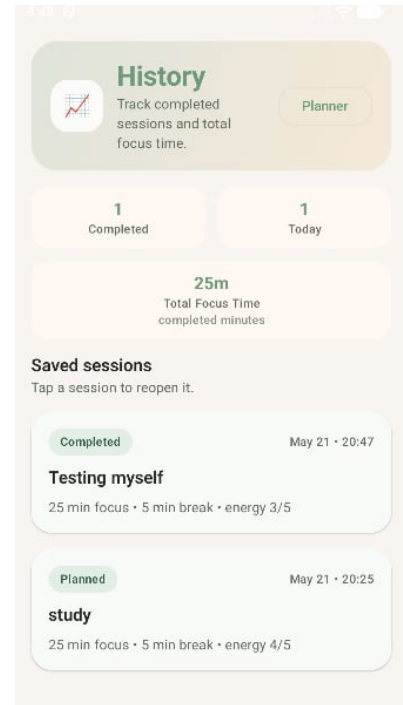
Figure 6: Timer workflow based on the Gemini recommendation

6.4 Summary and History

When the session is completed, the app updates the saved session and displays the result. The history screen retrieves previous sessions from Room Database. This is why data remains available after closing the application: it is stored in a local database file on the device, not only in temporary memory.



(a) Completed session summary



(b) Saved session history

Figure 7: Persistence and review features

6.5 Implementation Flow

The implemented flow is summarized below:

1. The user enters a task, selects energy, and selects available time.
2. `InputViewModel` validates the input and calls `AiRepository`.
3. `AiRepository` calls Gemini through `GeminiApiService`.
4. The response is converted into an `AiRecommendation`.
5. The generated session is saved through `SessionRepository` and `SessionDao`.
6. The user runs the timer and completes the session.
7. The session is updated locally and appears in summary/history.

7 Testing and Validation

Testing was performed manually on the emulator by running the complete user workflow. The test scenario used realistic tasks such as “Testing myself” and “study”. The goal was to verify that the application works from input to saved history.

Table 7: Manual validation checklist

Test area	What was checked	Result
Input validation	Empty task handling and correct input capture.	Passed
Gemini generation	Real task returns focus minutes, break minutes, tips, and motivation.	Passed
Timer controls	Start, pause, reset, break transition, and completion actions.	Passed
Local persistence	Planned and completed sessions remain visible in history.	Passed
Summary screen	Completed session details and AI recommendation are displayed.	Passed
Git safety	<code>local.properties</code> is excluded from the repository.	Passed

7.1 Observed Result

The MVP workflow is functional. The app can generate a plan, run the timer, save the session locally, show a summary, and load previous sessions in the history screen. The most important remaining technical improvement is to move the Gemini API key away from the Android client in a production deployment.

8 Limitations and Future Improvements

8.1 Current Limitations

The current version is designed as a functional MVP, not as a production-ready commercial application. The main limitations are:

- the Gemini API key is configured locally in the Android project during development;
- there is no authentication or user profile system;
- session feedback is not yet used to improve future prompts;
- analytics are limited to basic history and totals;
- automated unit and UI tests are not yet complete.

8.2 Future Work

Future improvements can extend the project without changing its main architecture:

- **Backend AI proxy:** call Gemini from a backend service instead of directly from the mobile client.
- **User profiles:** store preferred session lengths, goals, and recurring work habits.
- **Adaptive feedback:** let users rate a session and use this feedback in future prompts.
- **Notifications:** remind users about planned sessions.
- **Analytics dashboard:** show weekly focus time, streaks, and progress trends.
- **Automated testing:** add unit tests for repositories, Room tests for persistence, and Compose UI tests for screens.

9 Conclusion

FocusFlow+ demonstrates how AI can improve a simple productivity tool by adding context-aware planning. The app does not only start a timer; it first asks what the user wants to do, how much energy they have, and how much time is available. Gemini uses this context to generate a plan that is easier to execute than a fixed timer preset.

From a technical perspective, the project combines modern Android development with AI integration and local persistence. Jetpack Compose provides the interface, ViewModels manage screen state, repositories separate business logic, Retrofit connects to Gemini, and Room Database stores sessions locally. The result is a complete MVP that proves the main idea: a focus timer can become more useful when it adapts to the user's situation.

References

1. Android Developers. *Jetpack Compose documentation*. Available at: <https://developer.android.com/compose>
2. Android Developers. *ViewModel overview*. Available at: <https://developer.android.com/topic/libraries/architecture/viewmodel>
3. Android Developers. *Room persistence library*. Available at: <https://developer.android.com/training/data-storage/room>
4. Google AI for Developers. *Gemini API documentation*. Available at: <https://ai.google.dev/gemini-api/docs>
5. Square. *Retrofit documentation*. Available at: <https://square.github.io/retrofit/>